

AURENA

Do dado bruto ao Dashboard Power BI

Documentação executiva da ETL, integração com o IBGE, modelagem SQL e análise de inclusividade de tons

14	3	9	31
evidências técnicas	bancos de dados	recortes visuais do BI	expressões DAX

Objetivo

Apresentar, de forma clara e auditável, como os dados foram tratados, organizados no MySQL e transformados em indicadores para apoiar a leitura do mercado de bases.

Mensagem central: A Aurena conecta qualidade de dados, representatividade e oportunidade de mercado em uma única solução analítica.

AGOSTO DE 2026

Resumo executivo e fluxo de dados

Da coleta à tomada de decisão

O projeto organiza dados internacionais, nacionais e demográficos em um fluxo único. Cada etapa reduz inconsistências e prepara as informações para uso no Power BI.

- | | |
|-------------------|---|
| 1 ETL | Python e Pandas tratam nulos, duplicidades, textos e tipos de dados. |
| 2 IBGE | A API acrescenta população por cor ou raça, estado e região. |
| 3 MySQL | As tabelas relacionais organizam marcas, produtos, tons e dados demográficos. |
| 4 Power BI | Cards, rankings e comparativos transformam o banco em leitura executiva. |

Escopo técnico

- Dados cromáticos internacionais e módulo auxiliar de produtos e tons.
- Base nacional com marcas, produtos, características e tons.
- Dados oficiais do IBGE para comparação com a composição da população.
- Validação por contagens e regras DAX documentadas ao final.

Resultado: O processo cria uma base rastreável: é possível voltar do indicador exibido no dashboard até a tabela e o tratamento que originaram o valor.

1. Python e Pandas — tratamento inicial

Padronização dos dados antes da carga

```

Unificação das informações de identificação do tom

A análise mostrou que não existe nenhum registro com nome_tom e tom_especifico simultaneamente nulos. Portanto, todos os registros possuem pelo menos uma informação de identificação do tom.

Diante disso, foi avaliada a possibilidade de unificar essas informações.

As colunas possuem características diferentes: nome_tom contém nomes comerciais ou criativos atribuídos pelas marcas, sem uma padronização consistente entre os produtos, enquanto tom_especifico apresenta informações mais recorrentes e estruturadas para identificação do tom.

Como o objetivo das análises posteriores é trabalhar com uma única informação de identificação do tom, optei por criar a coluna tom_unificado. Nessa nova coluna, será priorizado o valor de tom_especifico e, nos casos em que ele estiver ausente, será utilizado o valor disponível em nome_tom.

Dessa forma, preservarei a informação disponível nos dados, reduzo a presença de valores nulos na variável que poderá ser utilizada para análise e mantenho as duas colunas originais para rastreabilidade da transformação.

# Cria uma coluna unificada aproveitando o nome_tom onde tom_especifico for NaN
df_shades["tom_unificado"] = df_shades["tom_especifico"].fillna(df_shades["nome_tom"])
print(df_shades[["nome_tom", "tom_especifico", "tom_unificado"]])

[118]
...
   nome_tom tom_especifico tom_unificado
0      NaN             355N           355N
1      NaN            100N           100N
2      NaN            110C           110C
3      NaN            120W           120W
4      NaN            130N           130N
...      ...             ...           ...
6811  Sand              5              5
6812  Dark             NaN           Dark
6813  NaN             Shade 11        Shade 11
6814  Naked Duck       NaN           Naked Duck

```

Unificação dos campos de tom

Função: Combina nome do tom e especificação quando um dos campos está vazio.

Resultado: Reduz valores nulos e preserva a informação disponível em uma coluna única.

```

#Verificando se deu certo e não temos nada nulo na coluna unificada
print(f"Nulos: {df_shades['tom_unificado'].isna().sum()}")

[119]
...
Nulos: 0

# Verificando se temos dados duplicados
df_shades.duplicated().sum()

[120]
...
np.int64(0)

```

Validação da qualidade

Função: Confere se ainda existem valores nulos e tons duplicados após o tratamento.

Resultado: Base limpa e pronta para relacionamento, análise e carga no banco.

Valor para o projeto: A limpeza evita que o mesmo tom seja contado mais de uma vez ou fique sem identificação.

2. API do IBGE — extração demográfica

Navegação no JSON e organização da resposta

```

15 def rodar_pipeline_sql():
16     """
17     Função principal que executa o Pipeline de Dados (ETL):
18     1. Extract (Extração): Busca dados populacionais direto da API do Censo 2022 (IBGE).
19     2. Transform (Transformação): Limpa, organiza e cria regras cosméticas nos dados.
20     3. Load (Carregamento): Cria a tabela e insere os dados no banco MySQL Workbench.
21     """
22     print("🚀 Iniciando o pipeline de dados do IBGE...")
23     print("1/3. Conectando à API do IBGE (Censo 2022)...")
24
25     # URL (endpoint) oficial do IBGE para a Tabela 9606 do Censo 2022
26     # Essa consulta traz a população distribuída por Cor ou Raça para cada Estado brasileiro.
27     url = "https://servicodados.ibge.gov.br/api/v3/agregados/9606/períodos/2022/variaveis/93?localidades=N3[all]&classificacao=86[all]"
28
29     # -----
30     # FASE 1: EXTRAÇÃO (FETCH DE DADOS NA API)
31     # -----
32     try:
33         # requests.get(): O Python envia uma solicitação para a internet buscando o endereço da API.
34         # timeout=15: Se o servidor do IBGE demorar mais de 15 segundos para responder, ele aborta.
35         response = requests.get(url, verify=False, timeout=15)
36
37         # O código de status 200 é o padrão do protocolo HTTP para "Sucesso / Deu Tudo Certo".
38         if response.status_code == 200:
39             dados_json = response.json() # Converte a resposta bruta do servidor em uma estrutura de dicionário Python
40             resultados = dados_json[0]['resultados']
41             lista_registros = [] # Lista vazia para ir guardando as linhas formatadas
42
43             # NAVEGAÇÃO NO JSON DA API:
44             # A resposta da API do IBGE é uma estrutura aninhada (listas dentro de dicionários).
45             # Precisamos "desempacotar" essas camadas para transformar em uma tabela simples.
46             for res in resultados:
47                 # Pega a categoria de Cor/Raça (ex: Branca, Parda, Preta, Amarela, Indígena)
48                 cat_dict = res['classificacoes'][0]['categoria']
49                 cat_id = list(cat_dict.keys())[0]
50                 cor_raca = cat_dict[cat_id]

```

Consulta e leitura da API

Função: Executa a requisição, percorre a estrutura retornada e identifica estado, cor ou raça e população.

Resultado: Tabela demográfica sem linhas agregadas de Total, evitando dupla contagem.

Funcionalidade: A API conecta a análise de tons ao tamanho real dos públicos em cada estado e região.

3. Enriquecimento e segurança do pipeline

Região, perfil de pele e continuidade do processamento

```

83 # PLANO DE CONTINGÊNCIA (FALLBACK):
84 # Se a internet cair ou o site do IBGE estiver fora do ar, o bloco except entra em ação.
85 # Ele cria uma pequena base mockada (estática) para o código não quebrar durante testes do time.
86 except Exception as e:
87     print("⚠️ Aviso ao conectar na API ((e)). Usando dados estáticos de contingência...")
88     df = pd.DataFrame([
89         ('Estado': 'São Paulo', 'Cor_Raca_Declarada': 'Branca', 'Populacao': 24700000),
90         ('Estado': 'São Paulo', 'Cor_Raca_Declarada': 'Parda', 'Populacao': 16100000),
91         ('Estado': 'São Paulo', 'Cor_Raca_Declarada': 'Preta', 'Populacao': 3800000),
92         ('Estado': 'Bahia', 'Cor_Raca_Declarada': 'Parda', 'Populacao': 8100000),
93         ('Estado': 'Bahia', 'Cor_Raca_Declarada': 'Preta', 'Populacao': 3100000),
94         ('Estado': 'Rio de Janeiro', 'Cor_Raca_Declarada': 'Branca', 'Populacao': 7200000),
95         ('Estado': 'Rio de Janeiro', 'Cor_Raca_Declarada': 'Parda', 'Populacao': 6300000),
96         ('Estado': 'Rio de Janeiro', 'Cor_Raca_Declarada': 'Preta', 'Populacao': 2300000),
97     ])
98
99 # -----
100 # FASE 2: TRANSFORMAÇÃO E ENRIQUECIMENTO DOS DADOS
101 # -----
102 print("2/3. Transformando e aplicando regras de negócio nos dados...")
103
104 # MAPA GEOGRÁFICO: Dicionário relacionando cada Estado à sua Região macro.
105 regioes = {
106     'Acre': 'Norte', 'Amapá': 'Norte', 'Amazonas': 'Norte', 'Pará': 'Norte', 'Roraima': 'Norte', 'Tocantins': 'Norte',
107     'Alagoas': 'Nordeste', 'Bahia': 'Nordeste', 'Ceará': 'Nordeste', 'Maranhão': 'Nordeste', 'Paraíba': 'Nordeste', 'Pernambuco': 'Nordeste',
108     'Distrito Federal': 'Centro-Oeste', 'Goiás': 'Centro-Oeste', 'Mato Grosso': 'Centro-Oeste', 'Mato Grosso do Sul': 'Centro-Oeste',
109     'Espírito Santo': 'Sudeste', 'Minas Gerais': 'Sudeste', 'Rio de Janeiro': 'Sudeste', 'São Paulo': 'Sudeste',
110     'Paraná': 'Sul', 'Rio Grande do Sul': 'Sul', 'Santa Catarina': 'Sul'
111 }
112 # Cria a coluna 'Regiao' mapeando os nomes dos estados
113 df['Regiao'] = df['Estado'].map(regioes).fillna('Outro')
114

```

Mapeamento geográfico

Função: Relaciona cada estado a uma das cinco regiões oficiais e prepara o cruzamento regional.

Resultado: Dados demográficos organizados para filtros de região e UF no Power BI.

```

115 # REGRA DE NEGÓCIO DERMOCOSMÉTICA:
116 # Função interna para classificar o perfil do consumidor com base na etnia e região climática.
117 def classificar_perfil_pele(row):
118     raca = row['Cor_Raca_Declarada']
119     regiao = row['Regiao']
120
121     # Classificação do fototipo (Escala Dermatológica de Fitzpatrick) e Subtom de Pele
122     if raca == 'Branca':
123         fototipo = 'II a III'
124         subtom = 'Neutro / Rosado' if regiao == 'Sul' else 'Amarelado / Neutro'
125     elif raca == 'Parda':
126         fototipo = 'IV a V'
127         subtom = 'Oliva / Amarelado'
128     elif raca == 'Preta':
129         fototipo = 'VI a VII'
130         subtom = 'Neutro / Quente'
131     elif raca == 'Amarela':
132         fototipo = 'III'
133         subtom = 'Amarelado / Quente'
134     else:
135         fototipo = 'III a IV'
136         subtom = 'Neutro'
137
138     # Textura recomendada de acordo com o clima/umidade característico da Região
139     if regiao in ['Norte', 'Nordeste']:
140         textura = 'Matte Alta Resistência' # Regiões mais quentes/úmidas
141     elif regiao == 'Sul':
142         textura = 'Hidratante' # Região mais fria
143     else:
144         textura = 'Soft Matte' # Regiões intermediárias
145
146     # Retorna os 3 valores em sequência
147     return pd.Series([fototipo, subtom, textura])
148

```

Classificação e contingência

Função: Aplica a regra de perfil de pele e usa dados de apoio se a API ficar indisponível.

Resultado: O desenvolvimento continua sem interromper o fluxo e sem perder a estrutura esperada.

4. Persistência no MySQL

Carregamento controlado da base demográfica

```

152 # -----
153 # FASE 3: CARREGAMENTO (CONEXÃO E PERSISTÊNCIA NO MYSQL WORKBENCH)
154 # -----
155 print("3/3. Conectando ao MySQL Workbench e salvando a tabela...")
156
157 # PARAMETROS DE CONEXÃO:
158 # Ajuste o valor da SENHA para a senha que você/sua colega criaram na instalação do MySQL.
159 USUARIO = "root" # Usuário padrão do MySQL
160 SENHA = "*****" # Coloque a senha local do seu MySQL Workbench aqui
161 HOST = "localhost" # Indica que o banco de dados está rodando na própria máquina
162 PORTA = "3306" # Porta padrão do MySQL
163 NOME_BANCO = "banco_beleza" # O Schema/Banco de dados criado no MySQL Workbench
164 NOME_TABELA = "demografia_pele_ibge"
165
166 # String de Conexão (URL de Conexão SQLAlchemy):
167 # Formato: mysql+pymysql://usuario:senha@host:porta/nome_do_banco
168 string_conexao = f"mysql+pymysql://{USUARIO}:{SENHA}@{HOST}:{PORTA}/{NOME_BANCO}"
169
170 # create_engine: É a ponte/motor de comunicação gerenciada pelo SQLAlchemy
171 engine = create_engine(string_conexao)
172
173 # df.to_sql(): Envia o DataFrame do Pandas direto para o MySQL
174 # if_exists="replace": Se a tabela já existir no banco, ele sobrescreve com os novos dados
175 # index=False: Não salva a coluna de índices numéricos (0, 1, 2...) do Pandas no banco
176 df.to_sql(NOME_TABELA, con=engine, if_exists="replace", index=False)
177
178 print("\n🌟 PIPELINE FINALIZADO COM SUCESSO!")
179 print(f"📊 Tabela '{NOME_TABELA}' criada e disponível no schema '{NOME_BANCO}' no MySQL Workbench!")
180
181 # Este bloco garante que o código só vai rodar se você executar este arquivo diretamente
182 if __name__ == "__main__":
183     rodar_pipeline_sql()

```

Conexão e gravação

Função: Usa PyMySQL e SQLAlchemy para conectar ao banco e gravar o DataFrame tratado.

Resultado: Tabela demografia_pele_ibge atualizada no schema definido para consumo analítico.

Conexão

Parâmetros do banco centralizados.

Carga

DataFrame enviado com df.to_sql().

Validação

Mensagem confirma a execução do pipeline.

5. Arquitetura do ecossistema SQL

Três bancos e três níveis de organização

A arquitetura separa os assuntos para manter o modelo simples, rastreável e adequado às análises do projeto.

shades_db	marcas_nacionais_db	ibge_db
Tons e produtos do mercado internacional.	Marcas, produtos e tons do mercado brasileiro.	População por estado, região e cor ou raça.

Modelo relacional

MARCA	PRODUTO	TOM
Fabricante único	Linha ou coleção	HEX, RGB, luminosidade e grupo

Leitura simples: A marca é a estante, o produto é o livro e os tons são as páginas com os detalhes usados nas métricas.

6. Estrutura SQL — mercado nacional

Tabelas relacionadas e integridade dos registros

```

Query 1  SQL File 2
Limit to 1000 rows

1  -- 1. Criação e seleção do banco de dados
2  CREATE DATABASE IF NOT EXISTS marcas_nacionais_db
3  DEFAULT CHARACTER SET utf8mb4
4  DEFAULT COLLATE utf8mb4_unicode_ci;
5
6  USE marcas_nacionais_db;
7
8  -- 2. Tabela de Marcas
9  CREATE TABLE marcas_nacionais (
10     id_marca INT PRIMARY KEY,
11     nome_marca VARCHAR(150) NOT NULL UNIQUE
12 );
13
14 -- 3. Tabela de Produtos / Linhas
15 CREATE TABLE produtos_nacionais (
16     id_produto INT PRIMARY KEY,
17     id_marca INT NOT NULL,
18     linha VARCHAR(255) NULL,
19     id_produto_marca VARCHAR(255) NOT NULL,
20     categoria_tom_marca VARCHAR(100) NULL,
21     peso_volum DECIMAL(10,2) NULL,
22     unidade VARCHAR(20) NULL,
23     cobertura VARCHAR(100) NULL,
24     acabamento VARCHAR(100) NULL,
25     textura VARCHAR(100) NULL,
26     tipo_pele VARCHAR(100) NULL,
27     vegano VARCHAR(100) NULL,
28     cruelty_free VARCHAR(100) NULL,
29     fps VARCHAR(50) NULL,
30     ativo_destaque VARCHAR(255) NULL,
31     CONSTRAINT fk_produtos_marcas FOREIGN KEY (id_marca) REFERENCES marcas_nacionais(id_marca) ON DELETE CASCADE,
32     CONSTRAINT uq_marca_produto UNIQUE (id_marca, id_produto_marca)
33 );
34

```

Marcas e produtos nacionais

Função: Cria as tabelas principais e conecta cada produto à marca por chave estrangeira.

Resultado: Reduz repetição e mantém as informações de cada linha vinculadas à marca correta.

```

35 -- 4. Tabela de Tons e Registros de Coleta
36 CREATE TABLE tons_nacionais (
37     id_registro INT PRIMARY KEY,
38     id_produto INT NOT NULL,
39     nome_tom VARCHAR(150) NULL,
40     subtom VARCHAR(150) NULL,
41     codigo_hex VARCHAR(10) NULL,
42     rgb_r INT NULL,
43     rgb_g INT NULL,
44     rgb_b INT NULL,
45     luminosidade_relativa DECIMAL(9,4) NULL,
46     luminosidade_pct DECIMAL(5,2) NULL,
47     grupo_luminosidade VARCHAR(100) NULL,
48     tipo_hex VARCHAR(255) NULL,
49     fonte_oficial TEXT NULL,
50     fonte_switch TEXT NULL,
51     image_switch_url TEXT NULL,
52     data_coleta DATE NULL,
53     status_qualidade TEXT NULL,
54     observacao_metodologica TEXT NULL,
55     fase_coleta VARCHAR(100) NULL,
56     versao_regua VARCHAR(100) NULL,
57     CONSTRAINT fk_tons_produtos FOREIGN KEY (id_produto) REFERENCES produtos_nacionais(id_produto) ON DELETE CASCADE
58 );

```

Tons e atributos

Função: Armazena cada tom com produto, nome, código, luminosidade e características comerciais.

Resultado: Base detalhada para filtros, comparação de portfólio e cálculo de inclusividade.

7. Estrutura SQL — módulo auxiliar

Organização de marcas, produtos e tons numéricos

```

1 • USE shades_db;
2
3 -- 1. Tabela de Marcas
4 • CREATE TABLE IF NOT EXISTS marcas_numbers (
5     id_marca INT PRIMARY KEY,
6     nome_marca VARCHAR(150) NOT NULL UNIQUE
7 );
8
9 -- 2. Tabela de Produtos
10 • CREATE TABLE IF NOT EXISTS produtos_numbers (
11     id_produto INT PRIMARY KEY,
12     id_marca INT NOT NULL,
13     nome_produto VARCHAR(255) NOT NULL,
14     id_produto_marca DECIMAL(10,1) NULL,
15     CONSTRAINT fk_produtos_marcas_numbers FOREIGN KEY (id_marca) REFERENCES marcas_numbers(id_marca) ON DELETE CASCADE,
16     CONSTRAINT uq_marca_produto_numbers UNIQUE (id_marca, nome_produto)
17 );
18
19 -- 3. Tabela de Tons/Shades
20 • CREATE TABLE IF NOT EXISTS tons_produtos_numbers (
21     id_shade INT PRIMARY KEY,
22     id_produto INT NOT NULL,
23     nome_tom VARCHAR(150) NULL,
24     tom_especifico VARCHAR(150) NULL,
25     tom_unificado VARCHAR(150) NULL,
26     luminosidade FLOAT NULL,
27     codigo_hex VARCHAR(10) NULL,
28     orden_luminosidade VARCHAR(10) NULL,
29     numero_tom FLOAT NULL,
30     CONSTRAINT fk_tons_produtos_numbers FOREIGN KEY (id_produto) REFERENCES produtos_numbers(id_produto) ON DELETE CASCADE
31 );

```

Tabelas do módulo _numbers

Função: Cria uma estrutura auxiliar com marca, produto e tom ligados por chaves estrangeiras.

Resultado: Mantém um recorte técnico separado; as análises centrais usam as tabelas principais do shades_db.

Governança: A separação evita misturar estruturas diferentes e reduz o risco de relacionamentos incorretos no Power BI.

8. Python para SQL — conexão e marcas

Primeira etapa da carga controlada

Logando do python para o SQL

```
import mysql.connector
import pandas as pd

# =====
# 1. CAMINHO DO CSV E CONFIGURAÇÃO DA CONEXÃO
# =====

CAMINHO_CSV = "D:.csv"

db_config = {
    'host':
    'user':
    'password':
    'database': 'shades_db',
    'charset': 'utf8mb4'
}

def limpa_val(val, tipo=str):
    if pd.isna(val) or val is None or str(val).strip().lower() == 'nan':
        return None
    try:
        return tipo(val)
    except (ValueError, TypeError):
        return None

try:
    conn = mysql.connector.connect(**db_config)
    cursor = conn.cursor()
    print("Conexão estabelecida com sucesso no shades_db!")
except mysql.connector.Error as err:
    print(f"Erro de Conexão: {err}")
    raise
```

Conexão

Abre a conexão com o banco e trata falhas antes da inserção.

Funcionalidade: O mapeamento de IDs mantém a ligação entre o arquivo de origem e os registros gravados no MySQL.

```
# =====
# 2. CARREGAMENTO E POPULAÇÃO
# =====

df = pd.read_csv(CAMINHO_CSV)

try:
    # =====
    # MARCAS (IDs gerados manualmente)
    # =====
    marcas_unicas = df["marca"].dropna().unique()
    mapa_marcas = {}

    for idx, nome_marca in enumerate(marcas_unicas, start=1):
        nome_marca_str = str(nome_marca).strip()
        id_marca_manual = idx # Gerando ID sequencial manual

        cursor.execute(
            """
            INSERT INTO marcas_numbers (id_marca, nome_marca)
            VALUES (%s, %s)
            ON DUPLICATE KEY UPDATE nome_marca = VALUES(nome_marca);
            """
            ,
            (id_marca_manual, nome_marca_str)
        )
        mapa_marcas[nome_marca_str] = id_marca_manual
```

Carga de marcas

Remove duplicidades, gera IDs e insere marcas com atualização controlada.

9. Python para SQL — produtos, tons e validação

Conclusão da carga relacional

```
# -----
# PRODUTOS (IDs gerados manualmente)
# -----
produtos_df = df[["marca", "produto", "id_produto_marca"]].drop_duplicates(subset=["marca", "produto"])
mapa_produtos = {}

for idx, (_, row) in enumerate(produtos_df.iterrows(), start=1):
    nome_marca = str(row["marca"]).strip()
    nome_produto = str(row["produto"]).strip()
    id_prod_marca = limpa_val(row["id_produto_marca"], float)
    id_marca_real = mapa_marcas[nome_marca]
    id_produto_manual = idx # Gerando ID sequencial manual

    cursor.execute(
        """
        INSERT INTO produtos_numbers (id_produto, id_marca, nome_produto, id_produto_marca)
        VALUES (%s, %s, %s, %s)
        ON DUPLICATE KEY UPDATE
            id_marca = VALUES(id_marca),
            nome_produto = VALUES(nome_produto),
            id_produto_marca = VALUES(id_produto_marca);
        """,
        (id_produto_manual, id_marca_real, nome_produto, id_prod_marca)
    )
    mapa_produtos[(nome_marca, nome_produto)] = id_produto_manual
```

Carga de produtos

Função: Associa cada produto ao ID correto da marca antes da inserção.

Resultado: Produtos gravados sem perder o vínculo com sua marca de origem.

```
# -----
# TONS / SHADES
# -----
for idx, row in df.iterrows():
    nome_m = str(row["marca"]).strip()
    nome_p = str(row["produto"]).strip()
    id_produto_real = mapa_produtos.get((nome_m, nome_p))

    if id_produto_real is None:
        continue

    id_shade_manual = idx + 1

    cursor.execute(
        """
        INSERT INTO tons_produtos_numbers (
            id_shade, id_produto, nome_tom, tom_especifico,
            tom_unificado, luminosidade, codigo_hex,
            ordem_luminosidade, numero_tom
        )
        VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
        ON DUPLICATE KEY UPDATE
            id_produto = VALUES(id_produto),
            nome_tom = VALUES(nome_tom),
            tom_especifico = VALUES(tom_especifico),
            tom_unificado = VALUES(tom_unificado),
            luminosidade = VALUES(luminosidade),
            codigo_hex = VALUES(codigo_hex),
            ordem_luminosidade = VALUES(ordem_luminosidade),
            numero_tom = VALUES(numero_tom);
        """,
        (
            id_shade_manual,
            id_produto_real,
            limpa_val(row["nome_tom"], str),
            limpa_val(row["tom_especifico"], str),
            limpa_val(row["tom_unificado"], str),
            limpa_val(row["luminosidade"], float),
            limpa_val(row["codigo_hex"], str),
            limpa_val(row["ordem_luminosidade"], str),
            limpa_val(row["numero_tom"], float)
        )
    )

conn.commit()
```

Carga de tons

Inserir os tons ligados ao produto, preservando luminosidade e demais atributos.

```
conn.commit()
print("Sucesso! Tabelas povoadas com sucesso.")

except mysql.connector.Error as err:
    conn.rollback()
    print(f"Erro no MySQL: {err}")

finally:
    if 'cursor' in locals():
        cursor.close()
    if 'conn' in locals() and conn.is_connected():
        conn.close()
```

Commit e encerramento

Confirma a transação, executa rollback em caso de erro e fecha a conexão.

Dashboard Power BI

Inclusividade de Tons

Leitura executiva dos indicadores, gráficos e expressões DAX

3	9	31	3+
áreas de análise	recortes visuais	DAX documentados	DAX por página

Objetivo

Mostrar a relação entre composição da população, oferta de tons no mercado internacional e desempenho das marcas nacionais.

Leitura: Os gráficos foram organizados para responder uma pergunta central: a oferta de bases acompanha a diversidade do público?

Resumo executivo do dashboard

Principais números e lógica de leitura

O painel reúne população, portfólios internacionais e marcas nacionais para medir amplitude, luminosidade e presença de tons escuros e retintos.



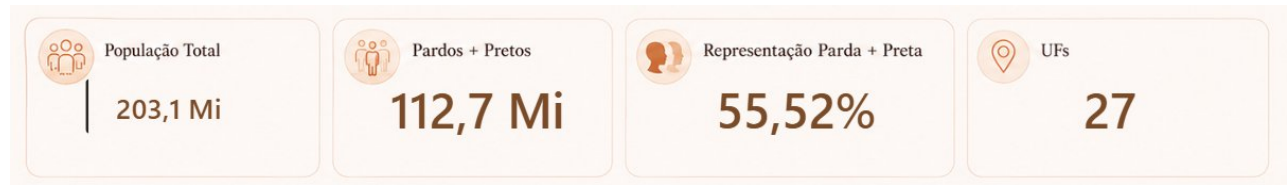
História apresentada pelos dados

- O público pardo e preto representa mais da metade da população analisada.
- O mercado internacional possui grande volume, mas menor participação proporcional de tons profundos.
- O mercado nacional é menor, porém apresenta participação mais próxima da composição do público.
- Quantidade de tons e profundidade da paleta devem ser avaliadas em conjunto.

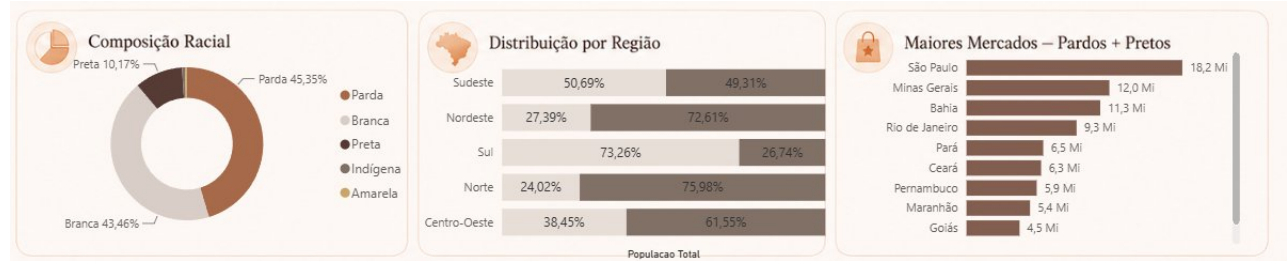
Uso executivo: Cards dão contexto; rankings mostram diferenças entre marcas; curvas e dispersão explicam como os tons estão distribuídos.

1. Visão Geral — Demografia

População, composição racial, regiões, estados e perfil de subtom



Leitura dos cards: A base analisada soma 203,1 milhões de pessoas. Pardos e pretos representam 112,7 milhões, ou 55,52% do total, com presença nas 27 UFs.



Composição e regiões: A composição é formada por 45,35% pardos, 43,46% brancos, 10,17% pretos, 0,60% indígenas e 0,42% amarelos. Norte (75,98%) e Nordeste (72,61%) têm as maiores participações de pardos e pretos; o Sul apresenta a menor (26,74%).

Maiores mercados: Em volume absoluto, São Paulo lidera com 18,2 milhões de pardos e pretos, seguido por Minas Gerais (12,0 mi), Bahia (11,3 mi) e Rio de Janeiro (9,3 mi).



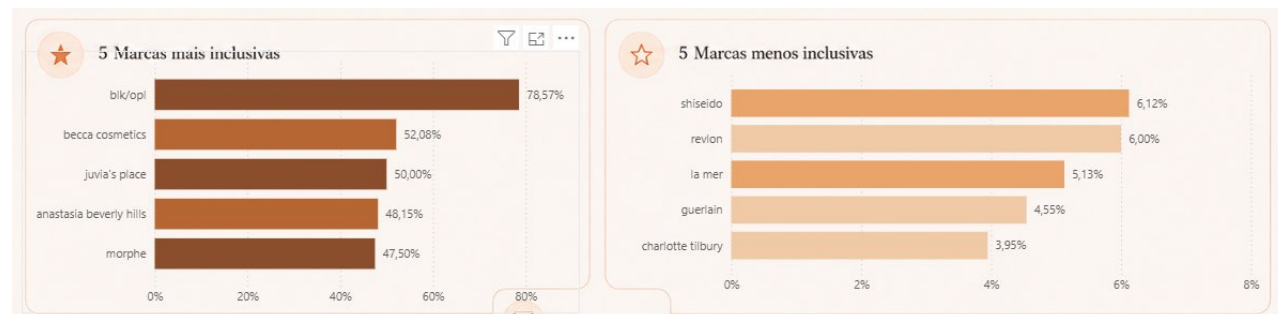
Subtom e tabela-resumo: O perfil concentra 92,1 milhões no grupo Oliva/Amarelado e 20,7 milhões em Neutro/Quente. A matriz detalha as participações raciais por região e funciona como conferência numérica dos gráficos, totalizando 100%.

2. Inclusividade de Tons

Mercado internacional, rankings e storytelling por produto



Cards: 107 marcas reúnem 6.816 tons, média de 63,7 por marca. Tons escuros e retintos representam 22,86% do portfólio.



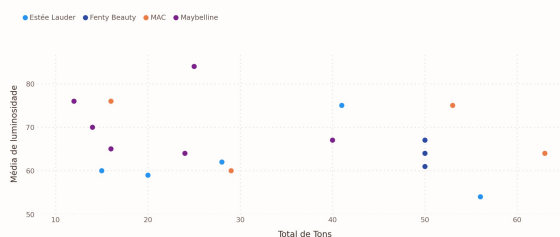
Ranking: O visual compara a presença proporcional de tons escuros e retintos. Para a regra oficial de elegibilidade, validar mínimo de 30 tons, faixa de 50% a 55% e exclusão de BLK/OPL.



Distribuição de luminosidade

A maior concentração ocorre em 60-69 e 70-79. Valores menores no eixo representam tons mais escuros.

FENTY BEAUTY x MARCAS TRADICIONAIS



Fenty x marcas tradicionais

Cada ponto é um produto: à direita há mais tons; mais abaixo, menor luminosidade média. A Fenty é comparada a Estée Lauder, MAC e Maybelline.

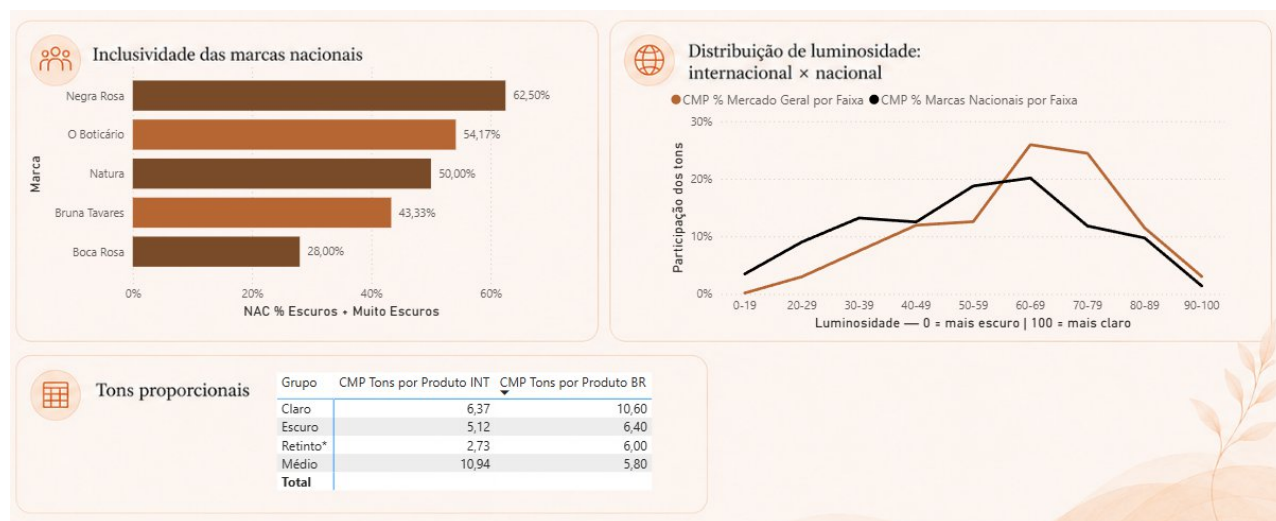
Storytelling: O gráfico corrigido mostra que quantidade de tons, sozinha, não garante profundidade; é necessário observar também a luminosidade média.

3. Mercado Nacional

Panorama das marcas brasileiras e comparação com o mercado internacional



Leitura dos cards: As cinco marcas nacionais analisadas reúnem 144 tons, média de 28,8 tons por marca e participação de 43,06% nos grupos escuro e retinto.



Inclusividade nacional: Negra Rosa lidera com 62,50%, seguida por O Boticário (54,17%), Natura (50,00%), Bruna Tavares (43,33%) e Boca Rosa (28,00%).

Distribuição comparada: A curva laranja representa o mercado geral e a preta as marcas nacionais. As nacionais têm maior participação nas faixas 0-59; o mercado geral se concentra mais entre 60-89. Na escala, 0 indica o tom mais escuro e 100 o mais claro.

Tons proporcionais: Por produto, o mercado nacional apresenta mais tons claros ($10,60 \times 6,37$), escuros ($6,40 \times 5,12$) e retintos ($6,00 \times 2,73$). O mercado internacional tem maior média no grupo médio ($10,94 \times 5,80$).

DAX — medidas 01 a 03

Print do código, finalidade e aplicação no dashboard

01. Populacao Total <pre>Populacao Total = SUM('Fato_Demografia'[Populacao])</pre>	VISÃO GERAL • CARD / BASE DOS GRÁFICOS PARA QUE SERVE Soma a população de todo o recorte ativo. É a base de comparação para composição racial, regiões, tabela-resumo e percentuais. USADO EM Cards, rosca, barras regionais e matriz
02. Populacao Parda <pre>Populacao Parda = CALCULATE([Populacao Total], 'Fato_Demografia'[Cor_Raca_Declarada] = "Parda")</pre>	VISÃO GERAL • MEDIDA AUXILIAR PARA QUE SERVE Isola a população declarada parda. Alimenta o cálculo combinado de pardos e pretos e os detalhes dos maiores mercados. USADO EM Tooltip e medida Parda + Preta
03. Populacao Preta <pre>Populacao Preta = CALCULATE([Populacao Total], 'Fato_Demografia'[Cor_Raca_Declarada] = "Preta")</pre>	VISÃO GERAL • MEDIDA AUXILIAR PARA QUE SERVE Isola a população declarada preta. É somada à população parda para formar o principal público representado no painel. USADO EM Tooltip e medida Parda + Preta

DAX — medidas 04 a 06

Print do código, finalidade e aplicação no dashboard

04. Populacao Parda + Preta	VISÃO GERAL • CARD / RANKING
<pre>Populacao Parda + Preta = [Populacao Parda] + [Populacao Preta]</pre>	PARA QUE SERVE Consolida os dois grupos de interesse. Serve para medir o tamanho do público e ordenar os maiores mercados estaduais. USADO EM Card e gráfico Maiores Mercados
05. % Parda + Preta	VISÃO GERAL • CARD / INDICADOR
<pre>% Parda + Preta = DIVIDE([Populacao Parda + Preta], [Populacao Total], 0)</pre>	PARA QUE SERVE Calcula a participação de pardos e pretos na população filtrada. É comparada ao total geral e permite leitura por UF ou região. USADO EM Card, distribuição regional e tooltips
06. Qtd UFs	VISÃO GERAL • CARD DE COBERTURA
<pre>Qtd UFs = DISTINCTCOUNT('Fato_Demografia'[Estado])</pre>	PARA QUE SERVE Conta quantas unidades federativas estão presentes no filtro. Confirma a cobertura nacional ou sinaliza quando há recorte ativo. USADO EM Card UFs

DAX — medidas 07 a 09

Print do código, finalidade e aplicação no dashboard

07. Total de Marcas

TONS E INCLUSÃO • CARD

```
Total de Marcas =  
DISTINCTCOUNT('shades_db marcas'[id_marca])
```

PARA QUE SERVE

Conta as marcas únicas disponíveis no contexto de filtro. É o denominador de cobertura do mercado internacional.

USADO EM

Card Marcas analisadas

08. Total de Tons

TONS E INCLUSÃO • CARD / BASE ANALÍTICA

```
Total de Tons =  
DISTINCTCOUNT('shades_db tons_produtos'[id_shade])
```

PARA QUE SERVE

Conta as tonalidades únicas catalogadas. Alimenta cards, distribuição de luminosidade, rankings e cálculos de participação.

USADO EM

Card, gráfico de luminosidade e tooltips

09. Média de Tons por Marca

TONS E INCLUSÃO • CARD

```
Média de Tons por Marca =  
AVERAGEX(  
    VALUES('shades_db marcas'[id_marca]),  
    CALCULATE(  
        DISTINCTCOUNT(  
            'shades_db tons_produtos'[id_shade]  
        )  
    )  
)
```

PARA QUE SERVE

Calcula a média dos catálogos individuais, respeitando cada marca. É mais adequado do que dividir diretamente tons por marcas.

USADO EM

Card Média de tons por marca

DAX — medidas 10 a 12

Print do código, finalidade e aplicação no dashboard

10. Tons Escuros

TONS E INCLUSÃO • MEDIDA AUXILIAR

```
Tons Escuros =
CALCULATE(
    [Total de Tons],
    'shades_db tons_produtos'[grupo_de_tom] =
        "Escuro / Dark"
)
```

PARA QUE SERVE

Conta os tons classificados como escuros. É combinada aos retintos para formar o indicador central de inclusividade.

USADO EM

Tooltips e participação inclusiva

11. Tons Retintos

TONS E INCLUSÃO • MEDIDA AUXILIAR

```
Tons Retintos =
CALCULATE(
    [Total de Tons],
    'shades_db tons_produtos'[grupo_de_tom] =
        "Retinto / Deep"
)
```

PARA QUE SERVE

Conta os tons mais profundos da classificação. É somada aos tons escuros para avaliar representatividade do portfólio.

USADO EM

Tooltips e participação inclusiva

12. Tons Escuro + Retinto

TONS E INCLUSÃO • MEDIDA CONSOLIDADA

```
Tons Escuro + Retinto =
[Tons Escuros] + [Tons Retintos]
```

PARA QUE SERVE

Consolida a oferta de tons escuros e retintos. É o numerador usado nos percentuais de inclusão das marcas.

USADO EM

Indicador e tooltips dos rankings

DAX — medidas 13 a 15

Print do código, finalidade e aplicação no dashboard

13. % Escuro + Retinto

TONS E INCLUSÃO • KPI

```
% Escuro + Retinto =
DIVIDE(
    [Tons Escuro + Retinto],
    [Total de Tons],
    0
)
```

PARA QUE SERVE

Mede a participação dos tons escuros e retintos no portfólio. Permite comparar marcas de tamanhos diferentes na mesma escala.

USADO EM

Card principal e base dos rankings

14. % Inclusividade Elegível

TONS E INCLUSÃO • RANKING SUPERIOR

```
% Inclusividade Elegível =
VAR TotalTons = [Total de Tons]
VAR Percentual = [% Escuro + Retinto]
VAR MarcaAtual =
    SELECTEDVALUE('shades_db marcas'[nome_marca])
RETURN
IF(
    TotalTons >= 30 && Percentual >= 0.50 &&
    Percentual <= 0.55 && MarcaAtual <> "blk/opl",
    Percentual, BLANK()
)
```

PARA QUE SERVE

Aplica os critérios de elegibilidade do projeto antes do ranking. Evita destacar catálogos pequenos ou resultados fora da faixa definida.

USADO EM

Gráfico 5 Marcas mais inclusivas

15. % Menos Inclusiva

TONS E INCLUSÃO • RANKING INFERIOR

```
% Menos Inclusiva =
IF(
    [Total de Tons] >= 30,
    [% Escuro + Retinto],
    BLANK()
)
```

PARA QUE SERVE

Mantém no ranking inferior apenas marcas com catálogo mínimo. É ordenada de forma crescente para encontrar os menores índices.

USADO EM

Gráfico 5 Marcas menos inclusivas

DAX — medidas 16 a 18

Print do código, finalidade e aplicação no dashboard

16. Faixa Luminosidade

TONS E INCLUSÃO • COLUNA CALCULADA

```
Faixa Luminosidade =
SWITCH(
  TRUE(),
  [luminosidade] < 20, "0-19",
  [luminosidade] < 30, "20-29",
  [luminosidade] < 40, "30-39",
  [luminosidade] < 50, "40-49",
  [luminosidade] < 60, "50-59",
  [luminosidade] < 70, "60-69",
  [luminosidade] < 80, "70-79",
  [luminosidade] < 90, "80-89",
  "90-100"
)
```

PARA QUE SERVE

Agrupa a luminosidade contínua em nove faixas. Torna a distribuição de tons mais fácil de ler e comparar.

USADO EM

Eixo do gráfico Distribuição de Luminosidade

17. Ordem Luminosidade

TONS E INCLUSÃO • COLUNA DE ORDENAÇÃO

```
Ordem Luminosidade =
SWITCH(
  [Faixa Luminosidade],
  "0-19", 1, "20-29", 2, "30-39", 3,
  "40-49", 4, "50-59", 5, "60-69", 6,
  "70-79", 7, "80-89", 8, "90-100", 9
)
```

PARA QUE SERVE

Força a ordem numérica correta das faixas. Sem ela, o eixo poderia ser classificado alfabeticamente.

USADO EM

Ordenação do eixo de luminosidade

18. Nome Marca Comparativo

TONS E INCLUSÃO • COLUNA AUXILIAR

```
Nome Marca Comparativo =
RELATED('shades_db marcas'[nome_marca])
```

PARA QUE SERVE

Leva o nome da marca à granularidade dos tons. Permite classificar e identificar cada ponto do comparativo.

USADO EM

Dispersão Fenty × marcas tradicionais

DAX — medidas 19 a 21

Print do código, finalidade e aplicação no dashboard

19. Marca Comparação

TONS E INCLUSÃO • COLUNA DE GRUPO

```
Marca Comparação =
SWITCH(
    TRUE(),
    [Nome Marca Comparativo] = "fenty beauty by rihanna",
        "Fenty Beauty",
    [Nome Marca Comparativo] = "mac", "MAC",
    [Nome Marca Comparativo] = "maybelline", "Maybelline",
    [Nome Marca Comparativo] = "estée lauder", "Estée Lauder",
    "Outras"
)
```

PARA QUE SERVE

Padroniza os quatro grupos mostrados no comparativo. É usada também como filtro do gráfico.

USADO EM

Legenda/eixo e segmentador do comparativo

20. Cor Destaque Fenty Coluna

TONS E INCLUSÃO • FORMATAÇÃO CONDICIONAL

```
Cor Destaque Fenty Coluna =
IF(
    [Marca Comparação] = "Fenty Beauty",
        "#C95B20",
        "#8D766A"
)
```

PARA QUE SERVE

Atribui uma cor de destaque à Fenty e uma cor neutra às marcas tradicionais. Facilita a leitura visual da comparação.

USADO EM

Cor dos pontos na dispersão

21. NAC Total de Tons

MERCADO NACIONAL • CARD

```
NAC Total de Tons =
DISTINCTCOUNT('df_nacionais_limpo 6(in)'[id_tom])
```

PARA QUE SERVE

Conta as tonalidades nacionais no contexto de filtro. Serve de base para os cards, o ranking por marca e as comparações com o mercado internacional.

USADO EM

Card, ranking e comparações nacionais

DAX — medidas 22 a 24

Print do código, finalidade e aplicação no dashboard

22. NAC Total de Marcas <pre>NAC Total de Marcas = DISTINCTCOUNT('df_nacionais_limpo 6(in)')[marca_ok])</pre>	MERCADO NACIONAL • CARD PARA QUE SERVE Conta as marcas nacionais únicas no contexto ativo. O indicador acompanha segmentações e confirma a abrangência da análise nacional. USADO EM Card Marcas analisadas
23. NAC Média de Tons por Marca <pre>NAC Média de Tons por Marca = AVERAGEX(VALUES('df_nacionais_limpo 6(in)')[marca_ok]), CALCULATE([NAC Total de Tons]))</pre>	MERCADO NACIONAL • CARD PARA QUE SERVE Calcula a média de tons por marca nacional respeitando os filtros. Permite comparar a amplitude média dos catálogos brasileiros. USADO EM Card Média de tons por marca
24. NAC Tons Escuros + Muito Escuros <pre>NAC Tons Escuros + Muito Escuros = CALCULATE([NAC Total de Tons], 'df_nacionais_limpo 6(in)')[grupo_luminosidade] IN {"Escuro", "Muito escuro"})</pre>	MERCADO NACIONAL • MEDIDA AUXILIAR PARA QUE SERVE Conta os tons nacionais classificados como escuros ou muito escuros. É o numerador do indicador de inclusividade nacional. USADO EM Tooltip, ranking e percentual nacional

DAX — medidas 25 a 27

Print do código, finalidade e aplicação no dashboard

25. NAC % Escuros + Muito Escuros

MERCADO NACIONAL • KPI / RANKING

```
NAC % Escuros + Muito Escuros =
DIVIDE(
    [NAC Tons Escuros + Muito Escuros],
    [NAC Total de Tons],
    0
)
```

PARA QUE SERVE

Compara a oferta inclusiva ao total de tons de cada marca nacional. Permite ordenar marcas de catálogos diferentes.

USADO EM

Card e gráfico Inclusividade das marcas nacionais

26. CMP Faixas Luminosidade

COMPARAÇÃO • TABELA CALCULADA

```
CMP Faixas Luminosidade =
DATATABLE(
    "Faixa", STRING, "Inicio", INTEGER,
    "Fim", INTEGER, "Ordem", INTEGER,
    {
        {"0-19", 0, 20, 1}, {"20-29", 20, 30, 2},
        {"30-39", 30, 40, 3}, {"40-49", 40, 50, 4},
        {"50-59", 50, 60, 5}, {"60-69", 60, 70, 6},
        {"70-79", 70, 80, 7}, {"80-89", 80, 90, 8},
        {"90-100", 90, 101, 9}
    }
)
```

PARA QUE SERVE

Cria um eixo comum e ordenado para comparar as distribuições internacional e nacional na mesma escala.

USADO EM

Eixo do gráfico internacional x nacional

27. CMP % Mercado Geral por Faixa

COMPARAÇÃO • LINHA INTERNACIONAL

```
CMP % Mercado Geral por Faixa =
VAR Inicio = MIN('CMP Faixas Luminosidade'[Inicio])
VAR Fim = MIN('CMP Faixas Luminosidade'[Fim])
VAR TonsFaixa =
    CALCULATE(
        [Total de Tons],
        FILTER(
            ALL('shades_db tons_produtos'[luminosidade]),
            [luminosidade] >= Inicio && [luminosidade] < Fim
        )
    )
RETURN DIVIDE(TonsFaixa, [Total de Tons], 0)
```

PARA QUE SERVE

Calcula a participação internacional em cada faixa de luminosidade. É comparada diretamente à curva nacional.

USADO EM

Linha Mercado Geral

DAX — medidas 28 a 31

Print do código, finalidade e aplicação no dashboard

28. CMP % Marcas Nacionais por Faixa

COMPARAÇÃO • LINHA NACIONAL

```
CMP % Marcas Nacionais por Faixa =
VAR Início = MIN('CMP Faixas Luminosidade'[Início])
VAR Fim = MIN('CMP Faixas Luminosidade'[Fim])
VAR TonsFaixa =
    CALCULATE(
        [NAC Total de Tons],
        FILTER(
            ALLSELECTED('df_nacionais_limpo 6(in)'),
            [luminosidade_pct] >= Início &&
            [luminosidade_pct] < Fim
        )
    )
RETURN DIVIDE(TonsFaixa, [NAC Total de Tons], 0)
```

PARA QUE SERVE

Calcula a participação nacional por faixa preservando a seleção de marca. Assim, a curva muda quando Negra Rosa ou outra marca é selecionada.

USADO EM

Linha Marcas Nacionais

29. CMP Grupos de Tons

COMPARAÇÃO • TABELA CALCULADA

```
CMP Grupos de Tons =
DATATABLE(
    "Grupo", STRING, "Ordem", INTEGER,
    {
        {"Claro", 1},
        {"Médio", 2},
        {"Escuro", 3},
        {"Retinto", 4}
    }
)
```

PARA QUE SERVE

Cria quatro categorias comuns para comparar quantidades proporcionais de tons internacionais e nacionais.

USADO EM

Linhas da tabela Tons proporcionais

30. CMP Tons por Produto INT

COMPARAÇÃO • TABELA PROPORCIONAL

```
CMP Tons por Produto INT =
VAR Grupo = SELECTEDVALUE(
    'CMP Grupos de Tons'[Grupo]
)
VAR TonsGrupo =
    SWITCH(
        Grupo,
        "Claro", CALCULATE(
            [Total de Tons],
            shades_db_tons_produtos[grupo_de_ton]
            = "Claro / Light"
        ),
        "Médio", CALCULATE(
            [Total de Tons],
            shades_db_tons_produtos[grupo_de_ton]
            = "Médio / Medium"
        ),
        "Escuro", [Tons Escuros],
        "Retinto", [Tons Retintos]
    )
VAR Produtos = DISTINCTCOUNT(
    shades_db_produtos[id_produto]
)
RETURN DIVIDE(TonsGrupo, Produtos, 0)
```

PARA QUE SERVE

Normaliza os tons internacionais pelo número de produtos. Isso evita comparar apenas volumes absolutos de bases diferentes.

USADO EM

Coluna INT da tabela Tons proporcionais

31. CMP Tons por Produto BR

COMPARAÇÃO • TABELA PROPORCIONAL

```
CMP Tons por Produto BR =
VAR Grupo = SELECTEDVALUE(
    'CMP Grupos de Tons'[Grupo]
)
VAR TonsGrupo =
    CALCULATE(
        [NAC Total de Tons],
        FILTER(
            'df_nacionais_limpo 6(in)',
            SWITCH(
                Grupo,
                "Claro", [grupo_luminosidade]
                IN ("Claro", "Muito claro"),
                "Médio", [grupo_luminosidade] = "Médio",
                "Escuro", [grupo_luminosidade] = "Escuro",
                "Retinto", [grupo_luminosidade] = "Muito escuro"
            )
        )
    )
VAR Produtos = DISTINCTCOUNT(
    'df_nacionais_limpo 6(in)'[produto]
)
RETURN DIVIDE(TonsGrupo, Produtos, 0)
```

PARA QUE SERVE

Normaliza os tons brasileiros por produto e grupo. É comparada à medida internacional na tabela-resumo.

USADO EM

Coluna BR da tabela Tons proporcionais

Conclusão executiva

Do tratamento dos dados à oportunidade de mercado

A Aurena estrutura um fluxo completo: trata os dados em Python, acrescenta informações do IBGE, organiza as tabelas no MySQL e transforma o resultado em indicadores no Power BI.

A análise aponta uma diferença entre público e oferta: pardos e pretos representam 55,52% da população analisada, enquanto tons escuros e retintos correspondem a 22,86% do mercado internacional. No recorte nacional, essa participação chega a 43,06%.

Recomendações

- Acompanhar quantidade de tons, luminosidade e participação de escuros e retintos em conjunto.
- Priorizar mercados com grande população parda e preta sem deixar de observar a participação relativa.
- Manter as tabelas principais e auxiliares separadas para evitar relações incorretas.
- Usar o ranking elegível somente após confirmar as regras de mínimo, faixa percentual e exclusões.
- Atualizar o dashboard com a mesma rotina de validação aplicada na carga e nas medidas DAX.

Entrega: O documento reúne evidências do pipeline, explicações dos gráficos e 31 prints DAX, mantendo o conteúdo dentro das margens e com identidade visual única.